MA 685

Zhenglin, Yiren, Lewen, Entong

Vacation Recommendation

AGENDA

01 Intro to Predictors

02 Data Preprocessing

03 Model Design

04 Two Added Methods

05 Model Ensemble

06 Model Performance

The Selected Ones

Introduce Predictors

Predictors for Users

1. Budget
2. Age
3. Ideal trip duration (days)
4. Current latitude + Current longitude
5. Climate preference
6. Travel style preference
7. Travel companion preference

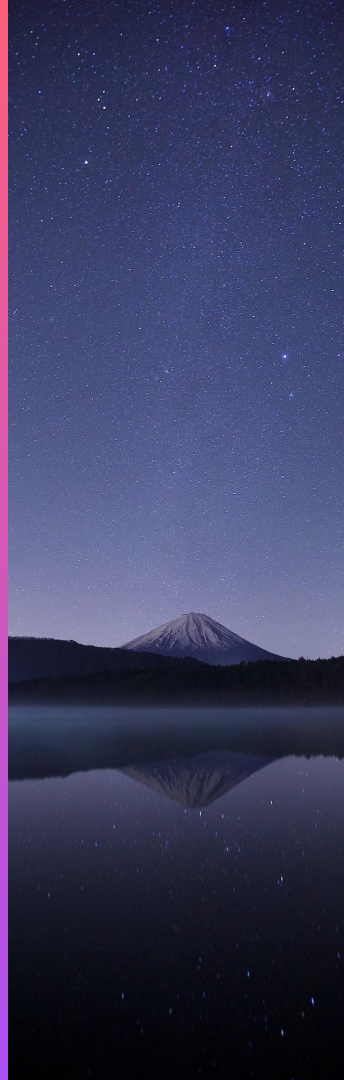
Predictors for Users :)

1. Parent education years: correlates with disposable income and cultural/educational trip preferences.
2. CEO annual salary: strong proxy for spending power and tolerance for premium destinations.
3. Favorite color: can proxy aesthetic preference
4. Local sports team record: indicates local mood/community engagement and can align with sports-related travel interest or desire for escape.

Predictors for Destination

1. City Name (destination)
2. Destination latitude + Destination longitude
3. AvgTemperature_C
4. Climate Zone
5. AvgCostOfLivingIndex
6. City Type

Data Preprocessing



Step 1: Train–test split

- shuffle all users with a fixed random seed
- 65% for training

```
set.seed(1234)
n = nrow(users)
ptrain = 0.65
ntrain = floor(n*ptrain)
idx = seq(1, n, 1)
itrain = sample(idx, ntrain, replace=F)
itest = setdiff(idx, itrain)
Xtrain = users[itrain,]
Xtest = users[itest,]
```

Step 2: Expanding past trips

- Each user originally has something like “Destination_12 – 8, Destination_7 – 5.”
- Pull out location ID and ratings
- One row per user to several trip rows

```
parse_trips = function(trips) {
  parts = unlist(strsplit(trips, ", "))

  loc = c()
  rat = c()
  for (p in parts) {
    output = unlist(regmatches(p, regexec("Destination_([0-9]+)\\s*-\\s*([0-9]+)", p)))
    loc = c(loc, as.integer(output[2]))
    rat = c(rat, as.integer(output[3]))
  }
  return(data.frame(LocationID = loc, Rating = rat))
}
```

Step 3: Combining user and destination information.

Step 4: Cleaning and selecting features

- User's information + destination's attributes
- User—destination pair with rating
- Removed columns not useful for prediction

Step 5: Encoding categorical variables

Step 6: Preparing for modeling

- Use model.matrix
- Convert categorical to one-hot encoded indicator
- The Rating column becomes Ytrain, our target vector
- Save the names and order of all feature columns

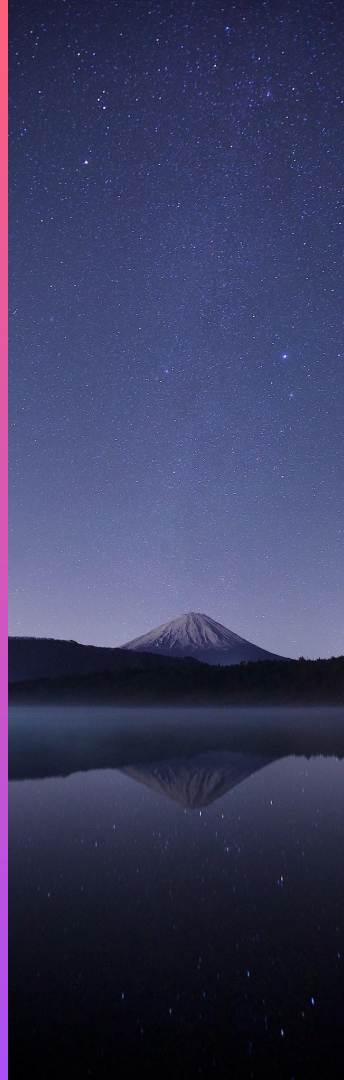
Step 7: Aligning future data

- Build align_matrix
- Ensures new data matches the layout of the training matrix

```
align_matrix = function(X_new, train_cols) {
  missing_cols = setdiff(train_cols, colnames(X_new))
  if (length(missing_cols) > 0) {
    add = matrix(0, nrow = nrow(X_new), ncol = length(missing_cols))
    colnames(add) = missing_cols
    X_new = cbind(X_new, add)
  }
  X_new = X_new[, train_cols, drop=F]
  X_new
}
```

Model Design

- Models Training
- Model Prediction for Ratings



KNN Regression

How to perform regression

1. Find the K nearest neighbors based on z-score standardization and Euclidean distance
2. Take the average rating of these K neighbors as the prediction
3. Repeat this process for each new sample.

```r
# 1) KNN average neighbor ratings
knn_rating = function(model, Xnew) {
  n_new = nrow(Xnew)
  ratings = numeric(n_new)

  for (i in 1:n_new) {
    KNN = knn(model$X, model$Y, Xnew[i, ],
              k = model$k, type = 2,
              ztype = "z-score")
    labs = as.numeric(KNN$k_nearest_labels)
    ratings[i] = mean(labs)
  }
  return(ratings)
}
```

Decision Tree Regression

How to perform regression

1. The tree splits the data into groups based on feature values.
2. Each group (leaf) stores the average rating of the training samples inside it.
3. For a new sample, we follow the tree's rules to find its leaf and use that leaf's average rating as the prediction.

```
# 2) DT regression
dt_rating = function(model, Xnew) {
  as.numeric(predict(model,
                     newdata = as.data.frame(Xnew)))
}
```

Random Forest Regression

How to perform regression

1. Each tree in the forest makes its own prediction based on its leaf's average rating.
2. The forest combines these predictions by taking their average.
3. The final averaged value is used as the predicted rating for the sample.

```
# 3) RF regression
rf_rating = function(model, Xnew) {
  as.numeric(predict(model, newdata = Xnew))
}
```

Supported Vector Machine Regression

How to perform regression

1. Train one binary SVM for each rating (1 vs rest, 2 vs rest…)
2. For a new sample, get the decision value from each SVM
3. Convert all decision values into probabilities using *softmax*
4. Compute the final prediction as the probability weighted average of the ratings.

```
# 4) SVM multi-rating wrapper
svm_multi_predict_one = function(model, myx) {
  ratings = model$ratings
  f = numeric(length(ratings))
  names(f) = ratings

  for (r in ratings) {
    out = model$models[[as.character(r)]]$predict_dplane(myx)
    f[as.character(r)] = as.numeric(out$fval)
  }

  f = f - max(f)
  p = exp(f) / sum(exp(f))
  return(sum(as.numeric(ratings) * p))
}

svm_multi_predict = function(model, Xnew) {
  apply(Xnew, 1, function(row) svm_multi_predict_one(model, row))
}

svm_rating = function(model, Xnew) {
  as.numeric(svm_multi_predict(model, Xnew))
}
```

SVM multi-rating wrapper

STRENGTHS

- Allows SVM to output continuous ratings
- Model structure more flexible

WEAKNESSES

Multiple Linear Regression

How to perform regression

1. Fit a linear model that maps features to the user's rating.
2. Plug the new sample's features into the linear equation.
3. The output is a continuous predicted rating.

```
# 5) MLR
mlr_rating = function(model, Xnew) {
  as.numeric(predict(model, newdata = data.frame(Xnew)))
}
```

K Mean Clustering Regression

How to perform regression

1. Cluster training points and compute the average rating for each cluster.
2. Assign each new sample to its nearest cluster.
3. Use that cluster's mean rating as the predicted score.

```r
# 6) KMC cluster mean rating
kmc_rating = function(model, Xnew) {
  km = model$kmc
  mr = model$mean_rate
  n_new = nrow(Xnew)
  ratings = numeric(n_new)

  for (i in 1:n_new) {
    cid = km$nearest_cids(Xnew[i, ])
    v = mr[as.character(cid)]
    ratings[i] = ifelse(is.na(v),
                        mean(mr, na.rm=T),
                        v)
  }
  return(ratings)
}
```

Motivation: Why We Needed New Methods

Problem: High-dimensional sparse one-hot features

1. Many categorical fields → hundreds of dummy variables
2. KNN: curse of dimensionality
3. Decision Trees: overfitting on sparse splits
4. Linear Regression: misses nonlinear interactions

Goal: Add learners that handle high dimension more effectively.

Why Naive Bayes Helps

Key strengths

1. Independence assumption factorizes high-dimensional likelihood
2. Stable parameter estimation even with sparse or limited data
3. Naturally low-variance model
4. Produces calibrated probabilities → useful for ensemble weighting

Role in ensemble: Robust baseline on sparse feature sets.

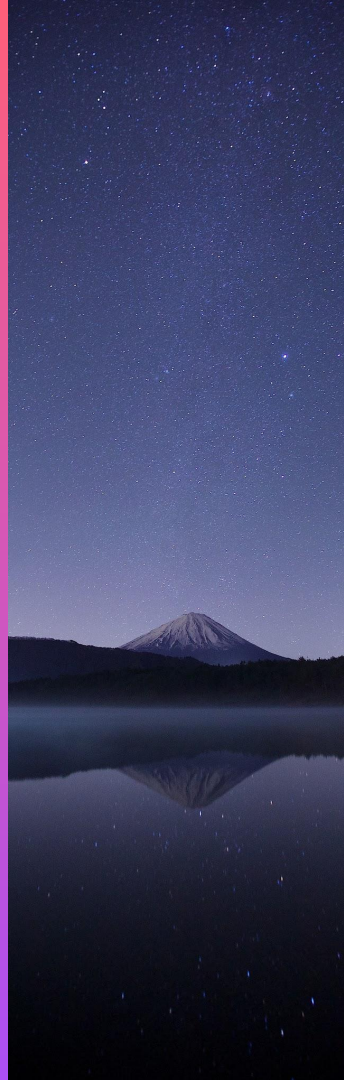
Why XGBoost Helps

Key strengths

1. Captures non-linear feature interactions
2. Regularization controls overfitting
3. Boosting reduces bias but keeps model compact
4. Feature subsampling lowers variance
5. Strong standalone accuracy → anchor model for ensemble

Role in ensemble: Provides high-capacity modeling and strong non-linear predictions.

Model Ensemble for Ratings



Weighted Voting Rating Ensemble

How to perform ensemble

1. Each of the eight base models predicts a rating for every user–destination pair.
2. The predictions are combined using predefined weights and normalized to sum to one.
3. The final ensemble score is a weighted average of all model outputs and is used to rank destinations.

Code For Ensemble

```r
### 4. weighted voting rating ensemble
soft_vote_predict_rating = function(ensemble, Xnew, weights = NULL) {
  models = c("knn","dt","rf","svm","mlr","kmc","nb","xgb")
  M = length(models)

  if (is.null(weights)) {
    weights = rep(1, M)
  }
  weights = weights / sum(weights)

  r1 = knn_rating(ensemble$knn, Xnew)
  r2 = dt_rating(ensemble$dt, Xnew)
  r3 = rf_rating(ensemble$rf, Xnew)
  r4 = svm_rating(ensemble$svm, Xnew)
  r5 = mlr_rating(ensemble$mlr, Xnew)
  r6 = kmc_rating(ensemble$kmc, Xnew)
  r7 = nb_rating(ensemble$nb, Xnew)
  r8 = xgb_rating(ensemble$xgb, Xnew)

  score = weights[1]*r1 + weights[2]*r2 + weights[3]*r3 + weights[4]*r4 +
          weights[5]*r5 + weights[6]*r6 + weights[7]*r7 + weights[8]*r8

  return(score)
}
```

Model Overall Pros & Cons

Pros

- Clear logic and highly interpretable model design
- Strongly diverse between models, more robust
- Simple weighted ensemble is intuitive and works reasonably well

Cons

Future Improvement

- Multivariate Linear Regression / Stacking
- Global Boosting (XGBoost meta model)
- Dimensionality reduction or embeddings

Performance

```
[[1]][[2]]
  LocationID PredRating        CityName Latitude Longitude AvgTemperature_C AvgCostOfLivingIndex ClimateZone CityType
1          7   7.066607 Destination_007  -9.6731 -129.8780             19.2                110.1         Dry    Urban
4         60   7.030280 Destination_060   3.2767 -153.2536             27.8                133.3    Tropical   Resort
3         37   7.027900 Destination_037 -22.3589 -125.1804             29.8                136.9    Tropical   Resort
5         61   7.025464 Destination_061  -8.8470 -135.4316             26.0                116.2         Dry    Urban
2         36   7.000859 Destination_036  21.7040 -121.0748             27.0                 79.3    Tropical  Natural


[[2]]
  LocationID PredRating        CityName Latitude Longitude AvgTemperature_C AvgCostOfLivingIndex ClimateZone  CityType
1         11   7.057046 Destination_011 -34.9729  -65.9747             11.1                109.8   Temperate  Cultural
3         20   6.990899 Destination_020  65.3395    7.0286             -9.1                127.4       Polar     Urban
2         13   6.988046 Destination_013 -71.3581   28.5933             -4.9                115.7       Polar  Cultural
5         65   6.930884 Destination_065  70.7214  121.5745             -4.2                 97.1       Polar Adventure
4         23   6.906214 Destination_023 -55.7815   77.1782             11.4                 81.1 Continental   Natural
```

Performance

```
> top5_no_info(Xtrain_all, dests, topk = 5)
  LocationID   Rating        CityName Latitude Longitude AvgTemperature_C AvgCostOfLivingIndex ClimateZone CityType
1         23 7.941176 Destination_023 -55.7815   77.1782             11.4                 81.1 Continental  Natural
5         64 7.897436 Destination_064  19.5634   94.8588             25.2                133.1         Dry    Urban
3         37 7.857143 Destination_037 -22.3589 -125.1804             29.8                136.9    Tropical   Resort
4         39 7.809524 Destination_039 -40.6049   73.2384             16.7                127.2   Temperate    Beach
2         33 7.805556 Destination_033  43.6099 -128.2623             12.5                153.9   Temperate    Urban
> Xtest[1:2,]
  UserID Age ParentalEducationYears CEOAnnualSalary  Budget IdealTripDurationDays CurrentLatitude CurrentLongitude ClimatePreference
1      1  25                   14.7        39719.36 1411.74                    5         40.8327         -74.8982              Warm
2      2  34                   13.3        70391.75 1217.15                    7         25.4330         -80.0823         Temperate
  TravelStylePreference TravelCompanionPreference FavoriteColor LocalSportsTeamRecord
1             Adventure                   Couples          Blue               Neutral
2                 Party                      Solo        Orange               Winning
                                                    PastTripsWithRatings
1 Destination_061-7, Destination_012-6, Destination_031-7, Destination_015-4
2                                       Destination_059-6, Destination_064-5
```

THANK YOU

Any questions or suggestions?